

Runbook

Dean Leon | 934513949
Containerized Minecraft Server
CS 312 System Administration

1. Runtime Architecture

The service is deployed on a t2.medium Amazon EC2 instance running Ubuntu. The Minecraft server is containerized using Docker and orchestrated via Docker Compose. Traffic flows from the internet through an AWS Security Group (allowing TCP/25565) to the host machine, which forwards it to the container. State is explicitly decoupled from the container lifecycle using a host bind mount.

```
[ Internet Clients ] --> (TCP: 25565)
      |
      [ AWS Security Group ]
      |
      [ EC2 Host (Ubuntu) ]
|-- Docker Port Binding (0.0.0.0:25565 -> 25565/tcp)
      |
      [ Minecraft Container ]
|-- Bind Mount (/data -> ~/minecraft-server/data)
      |
      [ EBS Persistent Storage ]
```

2. Container Deployment Definition

The service is defined in `docker-compose.yml`. It specifies the immutable image from the private ECR, sets runtime configuration via environment variables, and mounts persistent storage.

```
services:
  minecraft:
    image: ECR_URL.dkr.ecr.us-east-1.amazonaws.com/cs312-minecraft:mc-1.20.4-build1
    ports:
      - "25565:25565"
    environment:
      EULA: "TRUE"
      VERSION: "1.20.4"
      MOTD: "Dean Leon - CS312"
    volumes:
      - ./data:/data
    restart: always
```

3. ECR Usage & Image Tag Policy

We utilize a private Amazon Elastic Container Registry (ECR) to host immutable images. Deployments must pin a specific image tag; the `latest` tag is strictly prohibited in production deployments.

- **Image Provenance:** We re-tag the widely trusted `itzg/minecraft-server` base image. This upstream image is open-source, maintained by a highly active community, and provides reliable mechanisms for runtime version selection and EULA acceptance.
- **Tagging Scheme:** `mc-<version>-build<number>` (e.g., `mc-1.20.4-build1`). If runtime configurations change (such as pinning a different `VERSION=...`), a new build number or version number is issued to guarantee distinct deployable assets.

4. Build & Publish

To prepare a new version for deployment, the image must be tagged locally and pushed to the ECR.

```
# 1. Authenticate Docker to AWS ECR
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin
998487032255.dkr.ecr.us-east-1.amazonaws.com

# 2. Tag the trusted upstream image with our immutable scheme
docker tag itzg/minecraft-server:latest 998487032255.dkr.ecr.us-east-1.amazonaws.com/cs312-
minecraft:mc-1.20.5-build1

# 3. Push to ECR repository
docker push 998487032255.dkr.ecr.us-east-1.amazonaws.com/cs312-minecraft:mc-1.20.5-build1
```

5. Upgrade and Rollback Runbook

Pre-Change Checklist

- Backup world data to S3 prior to initiating any deployment changes.
- Verify the backup artifact exists and is visible in the S3 bucket.
- Verify both the target upgrade version and the known-good rollback version exist in the ECR repository.

Upgrade Execution

1. Edit `docker-compose.yml`. Update the `image:` line to the new tag (e.g., `mc-1.20.5-build1`) and update the `VERSION:` environment variable appropriately.
2. Apply the configuration: `docker compose up -d`

Post-Change Validation

- Run `docker compose ps` to confirm the container status is **Up**.

- Run `nmap -sV -Pn -p T:25565 localhost` to verify the port is open and the service is responding with the correct MOTD.
- Review logs using `docker compose logs --tail=50` for any fatal crash loops.

Rollback Execution

If the validation fails, immediately execute a rollback. Rebuilding from scratch is not required due to our immutable caching.

1. Edit `docker-compose.yml`. Revert the `image:` tag and the `VERSION:` variable back to the prior known-good state (e.g., `1.20.4`).
2. Apply the configuration: `docker compose up -d`
3. Repeat the Post-Change Validation steps.

6. Backup and Restore Runbook (S3)

Backups are stored in a private S3 bucket (`cs312-mc-backups-998487032255`). A bucket lifecycle rule expires artifacts older than 7 days. This 7-day retention period was chosen because it minimizes AWS storage costs while providing ample coverage for game-state rollback.

Backup Procedure

```
cd ~/minecraft-server
# Compress the host-mounted data directory
tar -czvf world-backup-$(date +%F).tar.gz data/
# Upload to S3
aws s3 cp world-backup-$(date +%F).tar.gz s3://cs312-mc-backups-998487032255/
```

Restore Procedure

```
cd ~/minecraft-server
# 1. Stop the active server to prevent file lock conflicts
docker compose down

# 2. Clear corrupted/current world data (use caution)
sudo rm -rf data/*

# 3. Retrieve the target backup from S3
aws s3 cp s3://cs312-mc-backups-998487032255/world-backup-TARGET.tar.gz .

# 4. Extract data in place
sudo tar -xzf world-backup-TARGET.tar.gz

# 5. Bring the service online
docker compose up -d
```

7. Video Demonstration

A video demonstration of the tasks can be found here: https://media.oregonstate.edu/media/t/1_tloxm5xl

Note: The video was edited and sections of down time, i.e., waiting for the ec2 instance to reboot, were cut out.

- **Checkpoint 1:** 00:06
- **Checkpoint 2:** 00:56
- **Checkpoint 3:** 01:26
- **Checkpoint 4:** 02:13

Document Attribution:

I used LLM assistance to format this document and to ensure that all CLI commands are correct.

<https://github.com/CS-312-001-S2026/minecraft-ops-GenericProgram>